Verifiable Recovery Across the Firmware–Kernel Boundary through a Hardware-Anchored Trust Handoff

Harshita Dhande
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
harshita.dhande@vit.edu.in

***Abstract*—Secure boot and measured boot are good at answering one question: did the machine start in a state we expected? They say much less about what happens afterwards, when a platform has to repair itself and then tell somebody, truthfully, that it has. In current designs firmware recovery and operating-system recovery are specified, built and audited by different groups, and the outcome of a recovery is usually reported by the same software that may have been compromised. This paper looks at that gap. We treat the boot path from the hardware root of trust to user space as one chain and ask four questions: which layer should start recovery, how firmware can notice a compromised kernel, whether a compromised kernel can fake a successful recovery, and how recovery status can cross layer boundaries safely. Our answer is a design we call the Recovery Attestation Chain (RAC): a hardware-anchored chain of measurements and keys, a small recovery controller that alone can write the golden images, and a nonce-bound token that the operating system can relay but cannot forge. We give a threat model, a protocol, a state machine and a qualitative analysis of six attack scenarios. The work is analytical; we have not built a prototype, and we list what an evaluation would need to measure.**

***Keywords*—firmware recovery, root of trust, measured boot, remote attestation, cross-layer security, UEFI, trusted computing**

I. Introduction

Most boot-path security work is about prevention. Secure boot refuses to run code that is not signed by a trusted party, and measured boot writes a hash of every stage into a tamper-resistant register so that someone can check the record later [1]–[3]. Both help. Neither tells a machine what to do after a check fails, or after a check is passed by something that later goes bad. The NIST platform firmware resiliency guidelines make the point directly by splitting the problem into protection, detection and recovery [4], and of the three, recovery is the least settled in practice.

Part of the trouble is organisational. Firmware and the operating system are studied by different communities and shipped by different teams. Firmware work asks how to keep the flash image intact; OS work asks how to keep the kernel honest. Attackers ignore that split. Several well-known BIOS and UEFI attacks began with software running inside the operating system and ended with a modified firmware image that survives a reinstall [5]–[7]. In the other direction, a kernel-level or hypervisor-level rootkit controls everything the operating system says about itself [8]. If recovery has to cross that boundary, then both directions matter, and the question of who is allowed to say "the system is healthy again" becomes a security question and not only a reliability one.

We study the whole chain from the hardware root of trust to user space (Fig. 1) and organise the paper around four research questions:

- *RQ1.* Which layer should initiate recovery?
- *RQ2.* How can firmware detect that the operating system has been compromised?
- *RQ3.* Can a compromised operating system falsely report that recovery succeeded?
- *RQ4.* How can recovery status be communicated securely across layers?

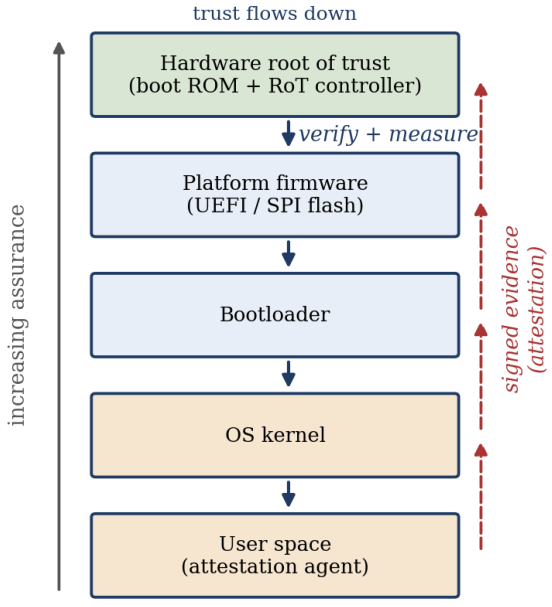


Fig. 1. Boot chain from the hardware root of trust to user space. Solid arrows show verification and measurement moving down; dashed arrows show signed evidence moving up toward a verifier.

The contributions here are modest. We write down a threat model for cross-layer recovery (Section III). We give reasoned answers to the four questions (Section IV). We propose a protocol and state machine, the Recovery Attestation Chain (RAC), in which the operating system can carry recovery

evidence but cannot produce it (Section V). And we walk through six attack scenarios to see where the design holds and where it does not (Section VI). Nothing has been built yet, so the analysis is qualitative; Section VII lists what a proper evaluation would measure.

II. Background and Related Work

A. Chains of trust at boot

The idea that each stage should verify the next before passing control goes back at least to AEGIS, which also proposed falling back to a known-good copy held in protected ROM when verification fails [1]. That early design has roughly the shape we want, but it works only at boot. It repairs a bad BIOS and then stops. Parno et al. later gave a broader account of how trust can be bootstrapped in commodity machines, covering the measurement, attestation and isolation primitives that later systems build on [2]. In current hardware the usual building blocks are a TPM that keeps measurements in platform configuration registers [3], an integrity measurement architecture that carries the idea into the kernel [9], and the UEFI specification that defines the firmware interfaces in between [10]. The DICE architecture is the closest match to our key-management needs: it derives a secret for each layer from a measurement of that layer, so a layer whose code has changed ends up with a different identity [11].

B. Firmware attacks and guidance

NIST SP 800-147 describes how BIOS should be protected against unauthorised modification [12], and SP 800-193 extends this to detection and recovery, with requirements such as a root of trust for detection and a protected recovery image [4]. The motivation for both is visible in attack research. Wojtczuk and Tereshkin showed how code running in System Management Mode could rewrite the BIOS from inside a live OS [5]. Bulygin et al. and Kallenberg et al. later showed Secure Boot being bypassed on shipping hardware through writable flash and unprotected variables [6], [7]. These papers show why recovery is needed. They say little about how a recovered platform convinces anyone else that it recovered.

C. Attestation and OS integrity monitoring

Remote attestation lets a verifier decide whether to trust a platform from signed evidence about its state [13]. Software-only schemes such as SWATT and Pioneer try to do this without special hardware and rely on timing assumptions [14], [15]; those assumptions are hard to keep on modern multi-core machines behind a jittery network, which is one reason we lean on hardware here. A different line of work watches the kernel from outside it. Copilot used a PCI coprocessor to read kernel memory without help from the host CPU [16], and virtual machine introspection moves the monitor into the hypervisor [17]. SubVirt is the cautionary tale in the opposite direction: malware that installs itself underneath the OS and so cannot be seen from inside [8]. Hardware enclaves such as SGX protect application code from a hostile OS [18], but they do not help when the thing being repaired is the firmware underneath.

D. What is missing

Existing work can verify state, and some of it can repair state. We did not find work that treats the recovery outcome itself as evidence, something that must survive a trip through software that may be hostile. RAC is aimed at that gap.

III. Problem Setting and Threat Model

A. Chain model

We model the platform as five layers L_0 to L_4: boot ROM and root-of-trust controller (L_0), platform firmware (L_1), bootloader (L_2), kernel (L_3) and user space (L_4). Before layer i hands control to layer i+1, it extends a running measurement held by the root of trust:

$$M_i = H(M_{i-1} \| H(C_i)) \quad (1)$$

Here H is a cryptographic hash, C_i is the code and configuration of layer i, and M_0 is fixed by the boot ROM. In parallel, each layer receives a secret derived from the one below it, in the style of DICE [11]:

$$K_i = \mathrm{KDF}(K_{i-1}, H(C_i)) \quad (2)$$

The point of (2) is that a layer's secret depends on what the layer is. If an attacker swaps C_3 for a modified kernel, the new kernel does not hold K_3, and nothing above it can reconstruct that key.

B. Assumptions

- The boot ROM and the root-of-trust (RoT) controller are correct and cannot be modified after manufacture.
- The RoT controller has private non-volatile storage for golden images and a monotonic counter, and no other component can write to either.
- The verifier knows the RoT's public attestation key from enrolment, for example through an endorsement certificate [3].
- The standard cryptographic primitives are secure.

C. Adversary and goals

The adversary can run arbitrary code at kernel privilege, including at hypervisor level [8], and can reach the flash interface through the OS as in the attacks of Section II-B. The adversary also controls the network between the platform and the verifier and may replay, delay or drop messages. We leave out invasive attacks on the RoT silicon, microarchitectural side channels and a malicious hardware vendor.

Two properties matter. *Soundness*: if a verifier accepts a recovery token, the layers named in it were running the measured code when the token was produced, and the recovery status in it was produced by the RoT controller. *Liveness* (best effort): if the platform is compromised and the RoT is reachable, the platform eventually either returns to a verified state or is quarantined, and the adversary cannot hold it in a state that merely looks healthy.

IV. Answering the Four Questions

A. RQ1: which layer starts recovery?

Whoever starts recovery needs three things. It must be at least as trusted as the layer being repaired, it must be able to

notice that something is wrong, and it must be able to write the repair. Table I compares the candidates on those terms.

TABLE I. CANDIDATE RECOVERY INITIATORS

Initiator	Visibility	Trust	Main weakness
RoT controller	Low: mailbox, heartbeat, hashes	Highest	Little semantic insight
Firmware	Medium: boot measurements	High until compromised	Itself a target [5], [7]
Bootloader	Medium: kernel image	Medium	Gone once the kernel runs
Kernel	High: full runtime state	Low if compromised	Can be silenced [8]
Remote verifier	Only what it is sent	Independent	Needs the network

No layer scores well on all three. The RoT controller is trusted and can write, but it sees very little. The kernel sees everything, but it is the thing under suspicion. Our answer to RQ1 is to split the job. Detection can come from anywhere, since a false hint only costs a recovery attempt, but the decision and the execution belong to the RoT controller. Put briefly: detect anywhere, decide low. A lower layer can always overrule a higher one that has gone quiet, and a higher layer can never cancel a lower layer's decision.

B. RQ2: how firmware notices a bad OS

Firmware cannot inspect a running kernel the way the kernel inspects itself, so it has to work from indirect evidence. We see three useful kinds. The first is boot-time evidence: if the kernel or its initial ramdisk does not match the signed manifest, the bootloader and firmware know before control passes. The second is liveness. A kernel-side agent has to answer a periodic challenge from the RoT within a deadline, and a missed deadline tells us something even though the content of an answer can never be trusted by itself. The third is out-of-band inspection, where hardware outside the host CPU's control reads kernel memory and hashes regions that should not change, in the manner of Copilot [16].

None of these is proof. Out-of-band monitors run into the semantic gap discussed in the introspection literature [17], legitimate updates change measured regions, and a careful rootkit can keep answering heartbeats. So we treat them as reasons to move the platform into a suspicious state, not as proof. There is a more important consequence. Firmware can never prove the OS is clean; it can only notice when the OS stops being able to show that it is. Recovery should therefore be defined on the absence of fresh good evidence and not on the presence of bad evidence.

C. RQ3: can the OS fake success?

In most deployed schemes, yes. Suppose the recovery routine ends by writing “recovered” to a UEFI variable or a log file that a management console reads later. A kernel with full privilege can write the same value without having recovered anything. A status flag is only as honest as the software that stores it.

In RAC the operating system is a courier and not an author. The status code R, which is one of none, firmware restored, OS restored or failed, is written by the RoT controller and goes into a signed token (Section IV-D). That leaves a compromised OS with three options. It can forge the token, which needs the RoT's key. It can replay an old one, which fails because the token binds a fresh verifier nonce and the monotonic counter. Or it can withhold the token, which the verifier sees as a timeout and treats as failure. So the design fails closed.

A subtler problem is time. A kernel could behave while it is being attested and turn bad a second later. The token only vouches for the instant it was produced. We shrink the window with short token lifetimes and periodic re-challenge, and we bind the token to a counter so that a silent reboot into a different state becomes visible, but we do not claim to close it. Section VI returns to this.

D. RQ4: crossing layer boundaries

Three mechanisms carry information across a boundary. Measurements flow down into the RoT through (1), so the RoT's record cannot be rewritten by the layer above. Identity flows through the key chain (2): a layer can only authenticate messages with a key that exists if its code is the expected code. And the link between a layer and the RoT is a mailbox in which every message carries a counter and a MAC under a session key derived from K_i. The mailbox needs integrity and freshness but not secrecy. A hostile kernel may read, delay or drop mailbox traffic; it should not be able to change it undetected. The token that leaves the platform is then

$$T = \mathrm{Sig}_{AK}(n \parallel M_k \parallel R \parallel c) \tag{3}$$

where AK is the RoT's attestation key, n is the verifier's nonce, M_k is the final measurement from (1), R is the recovery status and c is the monotonic counter value.

Table II pulls the four answers together, along with the mechanism in RAC that each one relies on.

TABLE II. SUMMARY OF ANSWERS TO THE RESEARCH QUESTIONS

RQ	Short answer	Mechanism in RAC
RQ1	The RoT controller decides; any layer may suggest	Two-stage SUSPECT and RECOVER, confirmed by the RoT
RQ2	Boot-time mismatch, missed deadlines and out-of-band hints; act on the absence of fresh good evidence	Challenge with a deadline; signed measurement manifest
RQ3	Not if the status comes from the RoT; the OS can only relay or withhold	Token T with nonce n and counter c
RQ4	Measurements down, keys derived per layer, authenticated mailbox	Equations (1) to (3) and a session MAC

V. THE RECOVERY ATTESTATION CHAIN

A. Components

RAC has five parts. The RoT controller is a small processor with its own firmware and storage, and it alone can write the protected flash region that holds golden images of the firmware, the bootloader and a minimal recovery OS. The monotonic counter lives inside it. The firmware and bootloader do what they do today and additionally extend measurements through

(1). The kernel runs an attestation agent that relays challenges and tokens. The verifier is a remote service, or in a simple deployment a local management controller, that issues nonces and checks tokens. We would like the RoT controller to be small enough to review properly; the smaller it is, the less the assumption in Section III-B asks of us.

B. Protocol

Figure 2 shows one attestation round. At boot the RoT verifies and releases the firmware (step 1), and the firmware verifies the kernel and extends the measurement (step 2). Once the system is up, the verifier sends a nonce *n* (step 3). The agent passes the request to the RoT (step 4), the RoT answers with a token as in (3) (step 5), and the agent relays it unchanged (step 6). If the token is missing, stale, or carries measurements that do not match the manifest, the verifier reports failure, and the RoT, which runs its own watchdog, can force recovery on its own (step 7). We include step 7 because a recovery that depended on the verifier alone would let a network attacker keep a compromised machine alive just by dropping the verifier's messages.

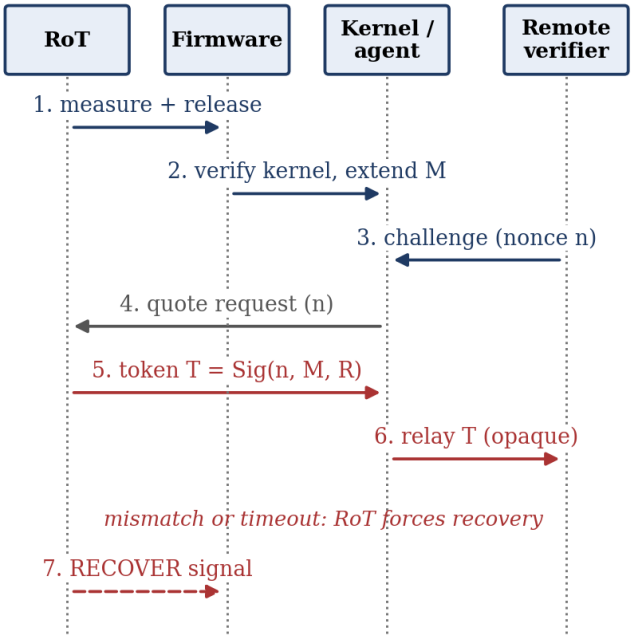


Fig. 2. One attestation round. Red steps carry the token T; the OS agent relays it but cannot alter it. On a mismatch or timeout the RoT forces recovery.

C. Recovery state machine

Figure 3 gives the platform's recovery state machine. The platform stays in NORMAL while challenges are answered correctly. A missed deadline, a hint from out-of-band monitoring or a complaint from the verifier moves it to SUSPECT. SUSPECT does not trigger repair by itself, which stops a flaky network from becoming a denial-of-service lever. The RoT moves to RECOVER only after it has confirmed the problem on its own, for instance through two consecutive failed challenges or a direct measurement mismatch.

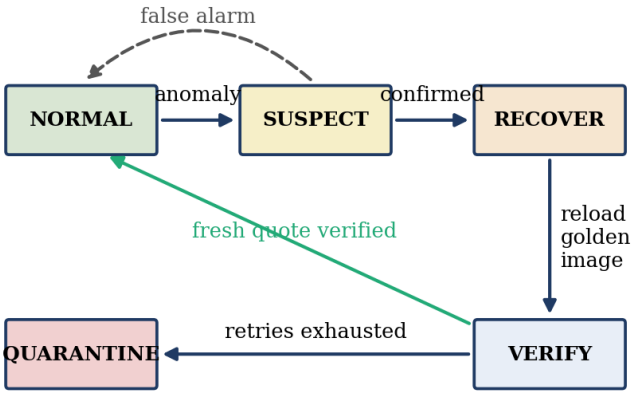


Fig. 3. Recovery state machine. Only the RoT can move the platform from SUSPECT to RECOVER.

What recovery does depends on the evidence. If the firmware measurement differs from the golden manifest, the RoT rewrites the firmware from the protected region, in line with the recovery requirements of SP 800-193 [4]. If only the bootloader or kernel differs, it reboots into the recovery OS and restores those images. In both cases the platform then enters VERIFY, where a fresh token with a new nonce must show the expected measurements and a recovery value of *R*. Success returns to NORMAL. Repeated failure exhausts a retry budget and sends the platform to QUARANTINE, where the RoT refuses to release sealed secrets (they stay sealed because the measurements no longer match [3]) and signals the verifier to isolate the host.

D. Rollback protection

One easy way to defeat a recovery scheme is to make it recover to something old and vulnerable. Golden images therefore carry a version number that is checked against the monotonic counter, and the counter moves forward only after a successful VERIFY following an update. The same counter appears in the token (3), so the verifier can notice a platform that has quietly gone backwards.

E. Relation to SP 800-193

RAC is meant to sit alongside the resiliency guidelines and not to replace them. SP 800-193 asks for a protected recovery image, a root of trust for detection and a way to restore firmware to a known-good state [4]. RAC keeps all three. What it adds is a reporting step that carries across the firmware and OS boundary: the recovery result becomes a signed, fresh statement that someone outside the machine can check. To our reading the guidelines concentrate on the firmware itself and give less direction on how that outcome should reach an owner who does not trust the host OS, which is the part we have tried to fill in.

F. A worked example

To make the moving parts concrete, consider a laptop that picks up a kernel rootkit through a vulnerable signed driver. The rootkit then tries to make itself permanent by writing to the SPI flash through the chipset's flash controller. Under RAC the active firmware region is write-locked once the firmware hands off, and any change request has to go through the RoT, so the

write is refused. An attacker who alters the image through some other flaw is caught at the next boot, because the measurement no longer matches the manifest.

Meanwhile the rootkit wants to stay hidden, so it kills the attestation agent or swaps in one that always says “healthy”. The first choice produces missed deadlines. The second is more interesting, because the RoT signs what it actually holds and not what the agent claims. If the rootkit was loaded through a driver that the kernel's own measurement list records [9], that driver shows up in the token and the verifier can reject it. If the rootkit left no trace in any measurement, only out-of-band checks can catch it. This is the limit we describe in Section VI, and we would rather state it than hide it. Suppose the rootkit is caught. After two failed challenges the RoT confirms the problem, reboots into the recovery OS from the golden image and restores the kernel. The platform enters VERIFY, the verifier sends a new nonce, and the RoT returns a token with the restored measurements and *R* set to “OS restored”. The verifier accepts, the counter moves forward, and the platform goes back to NORMAL. At no point did the operating system's own opinion of its health enter the decision.

VI. Security Analysis

We examine six scenarios. The comparison column describes a platform with secure boot and a TPM but without RAC, in which the recovery status is a flag kept by software. This is a reasoning exercise and not a measurement; Table III records what we conclude.

TABLE III. Attack scenarios with and without RAC

Scenario	Without RAC	With RAC
S1: firmware implant written from the OS	May survive reboots; flash writes not mediated	Flash writes go through the RoT; mismatch triggers RECOVER
S2: kernel rootkit hides itself	OS reports itself clean	Missed or wrong tokens move platform to SUSPECT
S3: forged “recovered” flag	Flag accepted	No valid token, status rejected
S4: replay of an old good token	Static log accepted	Nonce and counter mismatch
S5: rollback to an old golden image	Possible	Counter check refuses the image
S6: withheld or dropped messages	Silent failure	Timeout counts as failure; retry budget limits loops

Two points deserve a closer look. First, the main gain is in S3 and S4, the status-forging cases. These are not attacks on the boot chain itself. They are attacks on the story the platform tells afterwards, and that story is the part existing designs leave unprotected. Second, S6 shows the price of failing closed. A design that treats silence as failure hands the attacker a lever for causing repeated recoveries. The retry budget and the two-stage SUSPECT and RECOVER split are our answer, but the right thresholds depend on the deployment and we have no data with which to pick them.

A. Simpler alternatives

It is fair to ask whether all this machinery is needed. Three simpler designs are worth looking at. The first is to use the TPM alone. A TPM is a passive device: it records and signs measurements, but it cannot force a recovery or write a golden image, so it can tell a verifier that something changed without being able to do anything about it [3]. The second is to put the attestation agent in a hypervisor or a trusted execution environment, out of reach of a compromised guest kernel. That moves the trust to the hypervisor, and hypervisor-level rootkits are a known class of attack [8], so it narrows the problem without removing it. The third is software-based attestation, which needs no extra hardware but depends on tight timing assumptions that are difficult to guarantee in practice [14], [15]. Each of these is a sensible component of a larger system. What none of them gives on its own is an active, trusted party that holds the repair material and also signs the result.

B. Residual risks and cost

The remaining risks are real. The token only proves state at one instant, as discussed under RQ3. Everything rests on the RoT being correct, so a bug in its firmware, or a flaw in how it is manufactured or provisioned, undermines the whole chain. The mailbox is only as good as its implementation. And RAC does nothing for attacks that change no measured code, such as data-only attacks on the kernel, which a hash of a code image cannot see. As for cost, the extra work at boot is one hash extension per layer and one signature per attestation round. How that compares with a real boot-time budget, and how often attestation should run, are questions to measure and not to guess.

VII. Limitations and Future Work

A. Evaluation plan

The biggest limitation is that RAC is a design and not a system. A fair evaluation would proceed in stages. We would first build a software model of the RoT controller and mailbox under an emulator such as QEMU and run fault-injection campaigns that corrupt firmware regions, kernel images and mailbox messages. The numbers to report are time to detect, time to recover, boot overhead, and the rate at which healthy platforms are wrongly sent into recovery. A second stage would move the RoT onto an FPGA board or an open-silicon root of trust. In parallel, the token protocol is small enough to be checked in a symbolic tool such as ProVerif or Tamarin, which would support the replay and withholding arguments better than prose does.

Table IV lists the measurements we would collect in the first stage and what each of them is meant to tell us.

TABLE IV. Planned measurements for a prototype

Metric	Method	What it tells us
Time to detect	Corrupt the kernel image or silence the agent; time until SUSPECT	How fast a bad OS is noticed
Time to recover	Time from RECOVER to a verified NORMAL	Downtime cost of repair
Boot overhead	Extra boot time per layer against a baseline without RAC	Whether the scheme is deployable
False recovery rate	Healthy platforms under benign updates and network faults	How easily recovery is triggered by accident

Metric	Method	What it tells us
Protocol robustness	Replay, drop and reorder tests plus a symbolic model	Support for the claims in Section IV-C

B. *Deployment and open questions*

Deployment is its own challenge. In a data centre the verifier can be a fleet manager, and the baseboard management controller could host the RoT functions, although that controller then becomes a high-value target in its own right. On a laptop there may be no network at the moment recovery matters, so the platform needs a local policy for what to do when the verifier cannot be reached; our default of failing closed may be too strict for a user who is simply offline. Updates raise a similar tension. The golden image has to be replaceable, since vulnerabilities in it will be found, yet every update path is also an attack path. We think the counter-gated update described earlier is a reasonable starting point, but we have not examined the operational side.

Some questions stay open. System Management Mode and baseboard management controllers sit beside this chain and not in it, and both have their own history of vulnerabilities [5]. Multi-vendor update ecosystems blur who owns the golden image. Attestation tokens may be linkable across sessions, which raises privacy concerns we have not addressed. Finally, our adversary stops at the boundary of the RoT, and relaxing that boundary is a research problem of its own.

VIII. Conclusion

Firmware recovery and operating-system recovery are usually handled as separate engineering problems. A platform has only recovered, though, when the whole chain from the hardware root of trust to user space agrees that it has. We argued that the decision to recover should be made low in the stack, that evidence of compromise can come from many layers but should be treated as a hint, and that the status of a recovery should be produced by the layer that can be trusted and carried by the layer that cannot. The Recovery Attestation Chain is one way to do this, and our analysis suggests it closes the status-forging and replay gaps that simpler designs leave open. It does not remove the need to trust the root, and it has not yet been tested on real hardware. Those two facts decide what comes next.

References

[1] W. A. Arbaugh, D. J. Farber, and J. M. Smith, “A secure and reliable bootstrap architecture,” in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 1997, pp. 65–71.

[2] B. Parno, J. M. McCune, and A. Perrig, “Bootstrapping trust in commodity computers,” in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2010, pp. 414–429.

[3] Trusted Computing Group, “TPM 2.0 library specification,” Family 2.0, 2019. [Online]. Available: https://trustedcomputinggroup.org

[4] A. Regenscheid, “Platform firmware resiliency guidelines,” NIST Special Publication 800-193, National Institute of Standards and Technology, Gaithersburg, MD, USA, May 2018.

[5] R. Wojtczuk and A. Tereshkin, “Attacking Intel BIOS,” presented at Black Hat USA, Las Vegas, NV, USA, Jul. 2009.

[6] Y. Bulygin, A. Furtak, and O. Bazhaniuk, “A tale of one software bypass of Windows 8 Secure Boot,” presented at Black Hat USA, Las Vegas, NV, USA, Jul. 2013.

[7] C. Kallenberg, X. Kovah, J. Butterworth, and S. Cornwell, “Extreme privilege escalation on Windows 8/UEFI systems,” presented at Black Hat USA, Las Vegas, NV, USA, Aug. 2014.

[8] S. T. King, P. M. Chen, Y.-M. Wang, C. Verbowski, H. J. Wang, and J. R. Lorch, “SubVirt: Implementing malware with virtual machines,” in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2006, pp. 314–327.

[9] R. Sailer, X. Zhang, T. Jaeger, and L. van Doorn, “Design and implementation of a TCG-based integrity measurement architecture,” in *Proc. 13th USENIX Security Symp.*, San Diego, CA, USA, 2004, pp. 223–238.

[10] UEFI Forum, “Unified Extensible Firmware Interface (UEFI) specification,” Version 2.10, Aug. 2022. [Online]. Available: https://uefi.org

[11] Trusted Computing Group, “DICE layering architecture,” Version 1.0, Rev. 0.19, 2020. [Online]. Available: https://trustedcomputinggroup.org

[12] D. Cooper, W. Polk, A. Regenscheid, and M. Souppaya, “BIOS protection guidelines,” NIST Special Publication 800-147, National Institute of Standards and Technology, Gaithersburg, MD, USA, Apr. 2011.

[13] G. Coker *et al.*, “Principles of remote attestation,” *Int. J. Inf. Secur.*, vol. 10, no. 2, pp. 63–81, 2011.

[14] A. Seshadri, A. Perrig, L. van Doorn, and P. Khosla, “SWATT: Software-based attestation for embedded devices,” in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2004, pp. 272–282.

[15] A. Seshadri, M. Luk, E. Shi, A. Perrig, L. van Doorn, and P. Khosla, “Pioneer: Verifying code integrity and enforcing untampered code execution on legacy systems,” in *Proc. 20th ACM Symp. Operating Systems Principles (SOSP)*, Brighton, U.K., 2005, pp. 1–16.

[16] N. L. Petroni Jr., T. Fraser, J. Molina, and W. A. Arbaugh, “Copilot—a coprocessor-based kernel runtime integrity monitor,” in *Proc. 13th USENIX Security Symp.*, San Diego, CA, USA, 2004, pp. 179–194.

[17] T. Garfinkel and M. Rosenblum, “A virtual machine introspection based architecture for intrusion detection,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2003, pp. 191–206.

[18] V. Costan and S. Devadas, “Intel SGX explained,” IACR Cryptology ePrint Archive, Rep. 2016/086, 2016.